

Low-Level Design Document (LLDD)

Xyz Employee Management System

Introduction

- The **Xyz Employee Management System (EMS)** is designed to manage employees of XYZ Corporation across three categories: **Full-Time**, **Contractor**, and **Intern**.
- The system supports the following core operations:
- Adding employees (random or specific type)
- Removing (resigning) employees
- Searching employees by ID or name
- Converting employee types (Intern/Contractor → Full-Time)
- Managing leave details (for Full-Time employees)
- Maintaining employee status transitions (Active, Inactive, Resigned)
- The system uses a **custom Extended Doubly Linked List (EDLL)** for managing employee collections without relying on STL libraries.

Core Entities (Classes & Interfaces)

The system consists of the following core components:

XyzEmployeeIF (Interface) - Defines a contract that all employee types must follow using pure virtual functions.

Responsibilities:

- Provide getter methods for common attributes
- Provide setter methods for type-specific attributes
- Ensure abstraction and consistency across employee types

Common methods:

- `getEmployeeId()`
- `getEmployeeName()`
- `getEmployeeGender()`
- `getEmployeeType()`
- `getEmployeeStatus()`
- `getDOB()`
- `getDOJ()`
- `getDOL()`

Full-Time:

- `getTotalLeaves()`

- `setTotalLeaves()`
- `getAvailedLeaves()`
- `setAvailedLeaves()`

Contractor:

- `getAgencyName()`
- `setAgencyName()`

Intern:

- `getCollegeName()`
- `setCollegeName()`
- `getBranchName()`
- `setBranchName()`

XyzEmployee (Abstract Base Class) - Represents common properties shared across all employee types and implements the XyzEmployeeIF interface.

Common Attributes:

- Employee ID
- Name
- Gender
- Status
- Employee Type
- Date of Birth
- Date of Joining
- Date of Leaving

Responsibilities:

- ➔ Store shared employee information
- ➔ Provide common getter methods
- ➔ Implement XyzEmployeeIF interface
- ➔ Provide default implementations for type-specific methods
- ➔ Enable polymorphism

XyzFullTimeEmployee (Derived Class) - Inherits from XyzEmployee.

Additional Attribute:

- Leaves Availed - Number of leaves availed (Range: 0-22)
- mTotalLeaves - Total leaves allocated (Default: 22)

Responsibilities

- ➔ Track leave usage for full-time employees
- ➔ Override type-specific methods for leave management

XyzContractorEmployee (Derived Class) - Inherits from XyzEmployee.

Additional Attribute:

- External Agency - External contracting agency name

Responsibilities

- Store contractor agency details
- Automatically compute Date of Leaving as (DOJ + 1 year)
- Override agency-related methods

XyzInternEmployee (Derived Class) - Inherits from XyzEmployee.

Additional Attributes:

- College
- Branch

Responsibilities:

- Store intern academic information
- Automatically compute Date of Leaving as (DOJ + 6 months)
- Override college and branch methods

Data Structures

Node<T> - Represents a single node in the doubly linked list.

Attributes:

- mData (T) - Data stored in the node
- mPrev (Node<T>*) - Pointer to previous node
- mNext (Node<T>*) - Pointer to next node

Relationship:

- Composition with Deque<T>
- Node lifecycle depends entirely on Deque
- When Deque is destroyed, all Nodes are destroyed

Deque<T> (Base Class - Template) - Custom implementation of a double-ended queue using doubly linked list.

Protected Attributes:

- mHead (Node<T>*) - Pointer to first node
- mTail (Node<T>*) - Pointer to last node
- mCount (size_t) - Number of elements

Features:

- Insert at front (pushFront)
- Insert at rear (pushBack)
- Remove from front (popFront)
- Remove from rear (popBack)
- Access elements by index (operator[])
- Get size (size)
- Clear all elements (clear)
- Resize with direction (resize)
- Check if empty (isEmpty)
- Print all elements (printAll)
- Get first element (getFirstElement)
- Get last element (getLastElement)

Edll<T> (Extended Doubly Linked List) - Inherits from Deque<T>.

Purpose: Extends Deque with middle insertion/removal capabilities.

Additional Features:

- Insert at middle position (insertAtMiddle)
- Remove from middle position (removeAtMiddle)

Implementation Details:

- Index-based insertion/removal
- Delegates to parent class methods for front/back operations
- Handles boundary conditions properly

Relationship:

- Inherits from Deque<T>
- Composition with Node<T> (through Deque)

Manager Class

XyzEmployeeManager

Acts as the central controller of the system.

Data Members:

- mActiveInactiveEmpDeque (Edll<XyzEmployeeIF*>*) - Stores active and inactive employees
- mResignedEmpDeque (Edll<XyzEmployeeIF*>*) - Stores resigned employees
- mEmpIdCounter (int) - Counter for generating unique employee IDs

Responsibilities:

- Add new employees (specific type or random)

- Remove/resign existing employees
- Search for employees by ID or name
- Convert employees from Intern to Full-Time
- Manage leave allocation for full-time employees
- Maintain separate lists for active/inactive and resigned employees
- Memory management for all employee objects

Enumerations

Enumerations are used to enforce controlled values and improve type safety.

- **EmployeeStatus**

- ACTIVE
- INACTIVE
- RESIGNED

- **EmployeeType**

- FULL_TIME
- INTERN
- CONTRACT

- **Agency**

- AVENGERS
- JUSTICE_LEAGUE
- X_MEN

- **College**

- IIT_DELHI
- IIT_MUMBAI
- IIT_KANPUR
- IIT_HYDERABAD
- NIT_WARANGAL
- NIT_TIRUCHI
- IIIT_HYDERABAD

- **Branch**

- CSE
- CSIT
- ECE

Relationships in UML

The system defines the following relationships:

Inheritance

- `XYZFullTimeEmployee` → `XYZEmployee`
- `XYZContractEmployee` → `XYZEmployee`
- `XYZInternEmployee` → `XYZEmployee`
- `Edll<T>` → `Deque<T>`

Interface Implementation

- `XYZEmployee` implements `XYZEmployeeIF`

Composition

- `XYZEmployeeManager` → `Edll<XYZEmployeeIF*>` (owns two instances)
- `Deque<T>` → `Node<T>` (owns multiple nodes)

When `XYZEmployeeManager` is destroyed, all `Edll` instances are destroyed

When `Deque` is destroyed, all `Nodes` are destroyed

Behavioral Explanation

Adding an Employee

1. Manager generates employee details.
2. Create the appropriate derived object.
3. If `status == RESIGNED` → store in **resignedEmployees list**.
4. Else → store in **activeEmployees list**.

Removing an Employee

1. Search for the employee in the **activeEmployees list**.
2. Update status to `RESIGNED`.
3. Set the **Date of Leaving**.
4. Remove the employee from the active list.
5. Insert the employee into the **resignedEmployees list**.

Searching an Employee

1. Traverse the **activeEmployees list**.
2. If not found, traverse the **resignedEmployees list**.
3. Match employee using **ID or Name**.

Converting Employee Type (e.g., Intern → Full-Time)

1. Locate the **Intern object**.
2. Create a new **Full-Time Employee object**.
3. Copy all common attributes.
4. Remove the Intern from the list.
5. Insert the new Full-Time employee into the list.
6. Delete the old Intern object (memory cleanup).

Adding Leaves to All Full-Time Employees

1. Traverse the **activeEmployees list**.
2. For each Full-Time employee:
 - Increase leave count.
 - Ensure total leaves do not exceed **22**.